

# The QuaST Decision Tree: Achieving Automation With Data-Based Recommendations

Benedikt Poggel\*, Lena Tokuhira\*, Georg Kruse†, Jeanette Miriam Lorenz\*

\*Fraunhofer Institute for Cognitive Systems IKS

† Fraunhofer Institute for Integrated Systems and Device Technology IISB

{benedikt.poggel, lena.tokuhira, jeanette.miriam.lorenz}@iks.fraunhofer.de, georg.kruse@iisb.fraunhofer.de

**Abstract**—Quantum computers are increasingly powerful. Software tools for the development of quantum-enhanced algorithms are maturing. However, the software stack still lacks the connection to applications that would enable hybrid algorithms combining classical and quantum computing steps. End users need to be assisted in choosing the best combination of preprocessing, postprocessing, classical and quantum algorithms options. The application-facing software stack is therefore required to cover problem modeling, encoding, algorithm selection and hyperparameter tuning. A variety of tools exist for specific recommendations. The QuaST Decision Tree reflects the complexity in combining individual decisions in its modular network structure, consisting of flexible computation nodes with modular recommendations. It can easily be configured to serve in an industrial solver, an HPC software stack, or for rapid prototyping in development. The key ingredient, automation, is delivered by modules. We present one such module judging the feasibility of variational algorithms based on a robust scalability analysis and classification of problem instances. The automation improves the performance of end-to-end solutions, highlights the benefit to be gained from the hybrid quantum solution, reduces expensive trial-and-error testing, and leads to an improved utilization of quantum devices for a practical benefit.

**Index Terms**—quantum computing, hybrid quantum-classical algorithms, applied optimization, abstraction layer, automation, application-facing software stack

## I. INTRODUCTION

With quantum computing (QC) maturing, an increasingly relevant practical question is how to leverage the technology for the benefit of industrial applications. In particular, combinatorial optimization (CO) is enormously relevant to many industries such as logistics, production, and finance. Classical computing solutions typically employ a mixture of data preprocessing, exact algorithms and heuristics to tackle the high problem complexities [1], with the recent addition of reinforcement learning methods [2]. Hope for further improvement lies in QC with its inherent ability to represent large search spaces and to use quantum effects for a parallelized search, with advantages in solution quality (e. g., approximation ratios), time-to-solution and resource efficiency [3].

However, using QC for optimization is no small challenge: Many algorithms promise only a polynomial advantage in

computational complexity, bounded by Grover’s quadratic speedup [4]. Thus, for a real-world advantage, careful algorithmic design is needed, accounting for overheads from error correction, latencies, and iterations between classical and quantum computing. Even from the start, it must be determined which *problem instances* classical algorithms struggle with since a general advantage for entire classes of NP-hard problems is out of reach [3].

Moreover, applying quantum algorithms to optimization problems requires *hybrid algorithms* with complementing classical and quantum processing steps. The classical computer is not only required for data preprocessing and solution extraction in postprocessing, but also for decomposition steps needed to handle industrially relevant problem sizes with  $10^4$  and more variables. They divide the problem into parts to be treated classically and parts that benefit from QC. In this setting, QC solves specialized tasks just as a graphics processing unit (GPU) would do in a traditional setup. Finally, typical quantum algorithms themselves often require a quantum-classical iteration rather than the execution of a mere quantum circuit, e.g., variational quantum algorithms [5], or Quantum Branch-and-Bound [6].

Third, domain experts require abstraction software that allows them to turn their attention to the relevant parameters under their control and within their expertise: Modelling the problem, incorporating real-world constraints and objectives, and translating the solution of the mathematical model into the control language for their company processes. The amount of interdisciplinary expertise necessary to profit from quantum-enhanced solutions is significant and needs to be reduced through automation and abstraction.

The recent advances in QC hardware and improved accessibility have resulted in a wide variety of methods, tools and architectures aiming to provide abstractions to end users. High-level programming languages like Qrisp [7] or Qiskit [8] simplify handling quantum circuits and even hybrid algorithms, translating them into lower-level representations like OpenQASM [9] or QIR [10], and compiling and executing them. Specialized tools and solvers allow automating steps in the design and implementation process of a quantum-enhanced solution, e.g., by automating QC device selection [11] or hyperparameter tuning for specific quantum algorithms [12].

In existing tools, abstraction covers the mapping of a quantum algorithm to a QC hardware model [13]. With

the QuaST Decision Tree (QDT), the intention is to extend the quantum software stack to fully reach the end user at the application level by automating (quantum and classical) algorithm selection and the tasks necessary for it – problem encoding, hyperparameter tuning. Thus, the QDT goes beyond reducing implementation overhead for quantum algorithms. It provides a framework for *decision support* which recommends combinations of quantum and classical algorithms suited to a specific use case. Automation tools with this purpose are currently constrained to specific types of quantum algorithms, e. g., optimizing hyperparameters like the depth of a parameterized quantum circuit (PQC) in quantum approximate optimization (QAOA) [12], or orchestrating a specific hybrid algorithm, e. g. as Qiskit functions [14], [15]. An overarching framework with the ability to provide guidance across a wide range of quantum-enhanced algorithmic options is not within their scope. To reach this kind of abstraction in a research field as active as quantum algorithm research, an *extendable* framework is needed.

With this goal, we present the QuaST Decision Tree (QDT), a framework able to provide high-level abstractions for solving optimization problems, and demonstrate how automation can be reached with a module assessing the potential of concrete variational algorithms on specific problem instances. Our main contributions are

- the design principles, requirements and architecture layout of the automation framework. An intermediate status of these principles is available in [16].
- the implementation of the automation framework in the QDT core.
- an automation module assessing the potential of a set of variational quantum algorithms (VQAs) by classifying the problem instance and using a scaling analysis [17] from a database.

The remainder of this paper is structured as follows: Section II presents the necessary background and summarizes related work. Section III details design principles and requirements for the flexible automation framework. Then, its realization and implementation in the QDT is described: its structure (section IV), main components (section V), templates and input/output formats (section VI) and usage scenarios (section VII). Section VIII then describes the exemplary automation module based on a VQA scalability analysis [17]. Finally, the implications and relevance of our contributions are discussed, including next steps in developing the application-facing abstraction layer (section IX).

#### A. Notes on Implementation

The implementations are made in Python (Version 3.11) [18]. The notions of dictionaries, classes, instances, methods, attributes and objects are to be understood within the Python language (although a generalization would be possible). Abstract classes in this context refer to *class templates* where the exact realization of some or all methods is left to so-called *concrete* subclasses inheriting from the abstract class.

## II. BACKGROUND AND PREVIOUS WORK

For an overview on related work, section II-A starts with practical QC for optimization, and section II-B reviews existing approaches for application-facing abstraction. Section II-C summarizes the foundation for the scalability-based automation module. Finally, section II-D identifies the research gap addressed by the QDT and its modules.

### A. Practical Quantum Computing for Optimization

Due to its enormous relevance for industry, combinatorial optimization (CO) is an intensively researched field for quantum-enhanced algorithms. In the QC context, the standard case is *binary optimization*, in particular *quadratic unconstrained binary optimization* (QUBO) formulation with  $\mathbf{x}^* = \operatorname{argmin}_{\mathbf{x} \in \{0,1\}^n} \sum_{i,j=0}^{n-1} x_i Q_{ij} x_j$  where  $Q$  is called *QUBO matrix*. Many important CO problems can be mapped to QUBO, which can in turn be translated to an Ising model [19].

However, finding a suitable formulation in terms of binary variables is nontrivial especially in the presence of complex constraints, e. g. in vehicle routing [20] or scheduling [21].

The interest in applying QC to CO stems from its ability to explore large solution spaces in parallel, e. g. in Grover's algorithm [22] with a quadratic scaling advantage in computational complexity which is the best possible improvement for this problem type under standard assumptions [4]. Whether such an advantage is enough to beat classical algorithms has to be investigated on a case-by-case basis for a specific combination of problem characteristics and algorithm [3]. For this reason, applied research in QC-enhanced CO has led to a large variety of algorithms, typically using hybrid quantum-classical approaches for specific problem instances [3].

An important paradigm of hybrid QC for optimization are VQAs [5], especially variants of Quantum Approximate Optimization (QAOA) [23], [24]. They use a quantum-classical iterative loop where a parameterized quantum circuit (PQC) repeatedly evaluates the loss function of the CO problem and a classical optimizer is tasked with finding the circuit parameters minimizing the loss function. However, they are plagued by trainability issues like barren plateaus [25] and local minima [26]. Specific issues have been addressed in countless proposed variants [5].

Beyond noisy QC devices, quantum-enhanced algorithms for CO with error-corrected QC have been proposed where practical benchmarking is expected to become possible in the next years, with approximately 200 logical qubits foreseen by several vendors [27], [28], [29]. Promising ideas include, e. g., those based on Grover Adaptive Search, a variant of the Grover algorithm for CO [30], Quantum walk-based algorithms, and a quantum-enhanced Branch-and-Bound algorithm [6].

In terms of practical implementations, D-Wave's quantum annealer and hybrid solver offer solutions for hundreds and even thousands of binary variables [31]. On gate-based systems, commercial software implements algorithms, typically for a specific collection of backends (e. g., Qiskit functions for superconducting IBM backends [32]).

From an algorithmic side, central challenges are scaling to problem sizes with hundreds or thousands of variables which often requires the combination with classical decomposition methods, providing clear resource estimates for classical and QC resources, identifying promising applications, and managing the broad variety of algorithmic approaches outlined above to evaluate their practical benefits.

### B. Related Work in Abstraction Software

Higher abstraction levels for QC are an active research topic [13], [33], [34], [35]. Its primary focus is software development environments and high-level programming languages for convenient handling of quantum circuits. The diversity of applications, devices and stakeholders in the research field [36] is reflected in a large variety of QC-specific programming languages, some linked to specific vendors and technologies like the widely used Qiskit for superconducting QC [8], others following a more independent approach, e. g. Qrisp [7]. Common features are tools to define quantum circuits from individual gates or larger building blocks, with subsequent compilation for execution on hardware or simulated backends. Often, these steps, including the mapping to a hardware layout and circuit optimizations, are automated. However, end users still need the expertise to decide which algorithms and which quantum circuits to implement in order to fulfill their individual goals.

To support users in this regard, a recent push has come from design automation [37], [38]. Tools are provided, e. g., by the Munich Quantum Toolkit (MQT) comprising software for tasks like benchmarking, compilation, classical simulation, error correction, device selection and more [39]. The MQT Quantum Auto Optimizer [40], [41] is its user-facing abstraction tool for the CO domain. It employs supervised machine learning (ML) to select the best solver option from a given set based on QUBO input, combined with assistance in practical aspects like QUBO problem formulation.

Another approach handles the complexity of the selection of a quantum-enhanced solution by systematic exploration of the options. This meta-solving approach maps the task of selecting a good solution strategy to an optimization problem itself [42].

Apart from these and the earlier version of our QDT [43], no other approach exists, to our knowledge, for the challenge of selecting the best available algorithmic approach for CO problems from a general set (i. e., not variants of a single algorithm).

Beyond the circuit level, the commercial availability of quantum-enhanced solvers increases, as outlined in section II-A. Each relies on a specific algorithm for a fixed problem formulation (QUBO, HOBQ, or Linear Programming). The hyperparameter tuning is then performed automatically. Thus, end users do not need to concern themselves with aspects such as the mapping problem of a quantum annealer [31], error suppression for a custom QAOA variant [44], or the discretization and counterdiabatic acceleration of an adiabatic solution protocol [45]. While this greatly simplifies algorithm

setup and hyperparameter tuning, it leaves the fundamental question of which algorithm to choose unanswered.

### C. Scalability Analysis of Variational Quantum Algorithms

A fundamental question for variational quantum algorithms (VQAs) is whether their resource requirements scale favorably enough to achieve practical quantum advantage, in particular taking into account the ability of the classical optimizer to find the optimum. A systematic method characterizes the measurement shot requirements of VQA/optimizer combinations [17] and provides the empirical foundation for data-driven algorithm selection in the module presented in section VIII:

Two competing effects determine the feasibility of a VQA-optimizer combination for a given problem size  $n$ : First, the finite sampling error  $\varepsilon_{\text{FS}}(n, n_{\text{shots}})$  captures how measurement noise affects loss function evaluation, where  $n_{\text{shots}}$  denotes the number of measurement shots (circuit executions) used to estimate expectation values. This error scales as  $1/\sqrt{n_{\text{shots}}}$  for fixed system size, and grows exponentially in  $n$  at fixed shot count. Second, the empirical optimizer tolerance threshold  $\varepsilon^*(n)$  quantifies the maximum noise level under which the classical optimizer can still converge to a satisfactory solution. To extract it, optimizer success rates are measured and fitted across noise levels and converted to a normalized error measure using the variance of the CO loss function.

Crucially, the decline of  $\varepsilon^*(n)$  with system size follows VQA- and optimizer-dependent scaling laws. Across the tested configurations, exponential, power-law, and logarithmic decay functions are reasonable hypotheses. Additionally, problem structure influences the fitted parameters, necessitating problem-aware analysis. By combining both effects, the solvability condition  $\varepsilon_{\text{FS}}(n, n_{\text{shots}}) < \varepsilon^*(n)$  can be inverted to obtain the minimum shot requirement  $n_{\text{shots}}(n)$  for each VQA/optimizer combination under the different scaling hypotheses, with full error propagation providing uncertainty bounds on the estimates.

Finally, a hard feasibility criterion comes from the quantum disadvantage boundary: a quantum approach offers no benefit if its total computational cost  $n_{\text{shots}} \cdot n_{\text{calls}}$  exceeds classical brute-force enumeration at  $2^n$  evaluations. The number of quantum circuit calls  $n_{\text{calls}}$  depends on the optimizer. Any configuration where shot requirements exceed this boundary is outperformed by exhaustive classical search.

This methodology [17], thus, produces data for each analyzed VQA/optimizer combination: scaling parameters under all three hypotheses, shot requirement curves with propagated uncertainties, and feasibility classification across problem sizes. These results form the database underlying the automation module described in section VIII, enabling instance-specific resource estimation and algorithm recommendation within the QuaST Decision Tree.

### D. Missing Pieces for Reaching User-Level Abstraction

In conclusion, the following shortcomings require further advances, part of which are addressed in this work:

- The high complexity of the selection, setup and tuning of a quantum-enhanced solution is an obstacle for ML-based

solutions relying on large amounts of experimental data which is expensive to obtain. In this context, *quantum-enhanced* refers to algorithms combining QC-based and classical subroutines, both individually solving meaningful problem parts.

- Applied QC is a highly active research field, producing many algorithmic variants. However, individual results often cannot be easily combined without systematic knowledge of their interplay.
- To realize the best solution for an optimization problem, the broad selection of algorithmic variants needs to be accessible through a common abstraction layer.
- The decision taking process needs to be made efficient, without the need for extensive benchmarking and trial-and-error approaches.

To address these issues, we build on the initial proposal of a recommendation framework for quantum-enhanced optimization [43]. In comparison, we lay out a fundamental redesign of the QDT, replacing the layered architecture with a fully modular tree, including the Query mechanism to enable configurable automation. To emphasize the automation aspects, we demonstrate a concrete module to realize the vision of automated decisions based on a systematic scalability analysis of VQAs [17].

### III. DESIGN PRINCIPLES OF THE QUAST DECISION TREE

From the QC hardware, abstraction is built up in layers, with more and more of the underlying complexity being shielded in order to allow users to focus on higher-level aspects. Thus, a quantum software stack is manifesting. When moving to higher abstraction levels and closer to the application, some challenges arise and become more pronounced:

- 1) Non-technical personnel without expertise in the hidden complexity layers will increasingly come into contact with the quantum software stack. Even skilled developers (e. g., at a company providing specialized software services) typically have no detailed knowledge on QC devices and algorithms, hence this issue is amplified.
- 2) With the broader target group of end users in various industry sectors, reaching consensus on unified or even standardized approaches and enforcing the use of specific workflows, templates, and formats becomes harder. The software increasingly needs to be able to deal with incomplete, faulty or imprecise input.
- 3) Requirements from outside the quantum software stack increasingly influence which interfaces it needs to provide. To generate a benefit, QC-enhanced optimization solutions will need to integrate into diverse business workflows and professional software environments, creating a wide set of industry-relevant usage scenarios with business-related requirements to the data flow, data availability, time constraints and desired output.

At the inception of the QDT, the different processing steps from problem formulation to quantum algorithm execution were organized in layers: problem formulation and decomposition, encoding, algorithm selection, classical optimizer

selection and compilation/hardware selection deferred to the lower-level software stack [43]. Since this approach lacks flexibility and imposes impractical constraints on the available algorithms, the final QDT in this work features a fully modular setup for maximal flexibility and adaptivity. The backbone for this systematic approach remains a *decision tree* structure, capturing and organizing the rich options to be made in the setup of a quantum-enhanced solution.

The vision of the QDT is to assist users by automating the decision between these options and providing recommendations optimized for application-specific quantum-enhanced algorithm performance. For sustainable benefits, the framework needs to be efficiently *usable*, *expandable* and *maintainable*, covering the different usage scenarios in quantum-enhanced optimization (see section VII). This vision leads to a set of design principles:

- 1) *Automation without Restriction*: A convincing *killer application* [46] from the technology and business perspective is lacking. Therefore, abstraction software needs to stay open to innovation and avoid restricting the available exploration space for new algorithms and methods. Expert users should be able to understand the framework, retrace its decisions, and implement solution paths deviating from the recommendations.
- 2) *Modularity*: New modules can be inserted easily. The primary types of modules are automation subroutines, third-party software and adapters for the different usage scenarios.
- 3) *Locality*: To keep the maintenance of the framework feasible, adding or removing modules should require modifications only in modules that interact directly (e.g., by building on each other). Anticipating section IV, the locality will be realized by organizing the decision tree into *nodes* performing the individual computing tasks. To capture the complexities and interdependencies in the decision tree, locality cannot be entirely fulfilled at every point – for example, even high-level nodes often require information about the available hardware backends before deciding on a problem encoding.
- 4) *Configurability*: A central configuration source (e.g., a YAML file) should allow modifying the behavior of individual decision tree nodes and the full decision tree, notably controlling the degree of automation, and strategies used to derive recommendations.
- 5) *Efficiency*: The entire quantum-enhanced workflow realized by the framework needs to come with minimal overhead. In applications, the end-to-end performance metric will include the processing time within the framework.
- 6) *Transparency*: Data processing in the framework should be transparent, revealing what data is transmitted, where, how, and to what purpose it is modified.

### IV. OVERVIEW OF THE QUAST DECISION TREE

The QDT is based on the design principles (section III). It defines a framework that can be used for different purposes in the solution pipeline starting at the application. Its unique

strengths are the ability to incorporate third-party software, and to adapt to its execution environment, be it a QC software stack at a HPC centre, a commercial solver, or standalone. The implementation in Python [18] is a compromise to simplify scientific contributions at the expense of implementation efficiency. This shortcoming can easily be mitigated by leveraging, e.g., C-based modules for performance-critical components.

The QDT is provided in the `quast-decisiontree` package whose subpackage structure is shown in fig. 1. The central Python object, the `DecisionTree` class and its components are contained in the `core` subpackage. The remaining subpackages `algorithms`, `nodes`, `problems` and `utils` provide necessary ingredients for a basic instance of the `DecisionTree` class which can illustrate and demonstrate its functionality while serving as a starting point for automation modules and concrete usage scenarios. The main automation and recommendation capabilities as well as adapters for different environments, e.g., the Munich Quantum Software Stack (MQSS) [47], are then provided as plugins. One such automation module is described in section VIII.

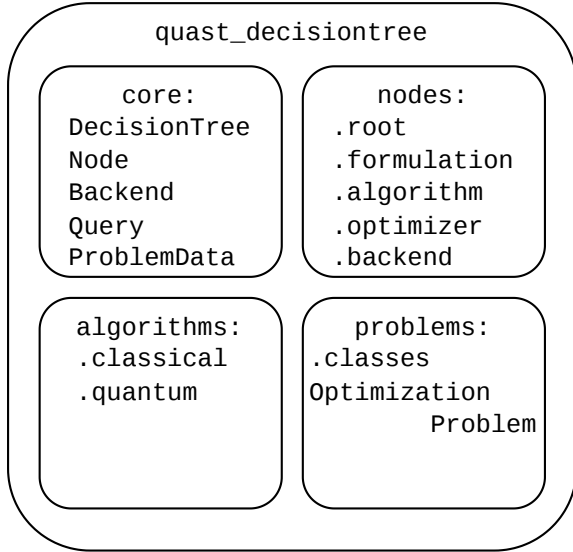


Figure 1: Components of the `quast-decisiontree` package. `core` contains the framework, the `DecisionTree` base class and its building blocks. `nodes` provides a basic instance covering steps from problem loading to algorithm execution. `algorithms` and `problems` define templates for algorithms and problem classes, respectively.

The `core` part comprises the following components, each realized as Python classes:

- **DecisionTree** orchestrates the setup and execution of the QDT instances.
- **Nodes** are the constituents of the decision tree, able to flexibly execute atomic computational tasks.
- **ProblemData** is the centralized data structure passed between the nodes of the decision tree.

- **Queries** enable different levels of automation and serve as decision points. A global configuration flag enables either the automated execution, or manual option selection.
- **Backends** are integrated via specialized nodes tasked with the communication with the backend (circuit definition, submission, and result retrieval).

The detailed functioning of the QDT follows in section V, including the discussion on how the design principles from section III are satisfied. A no-code instance is available online for exploration and testing [48].

From a high-level standpoint, the intended workflow with the QuaST Decision Tree is:

- 1) Create an *instance* of the QDT by specifying the nodes to include and their order.
- 2) *Configure* the behavior of the instance via a configuration file or by setting flags manually.
- 3) *Run* the QDT instance on the desired problem instance with the given configuration, by invoking its *run* method. This triggers a *forward pass* through the decision tree, then reverses for a *backward pass*. The typical breaking point between the passes is the submission and execution of the quantum algorithm, with algorithm setup and preprocessing on the forward pass and postprocessing, decoding and result interpretation on the backward pass.

The output of the QDT depends on its configuration. One QDT instance can be used for multiple problem instances with the same configuration. E.g., an administrator can create and configure the QDT instance and allow users to execute runs.

## V. MAIN COMPONENTS OF THE QUAST DECISION TREE

The detailed workflow realized by the QDT is laid out from the bottom up, first describing data handling and the vital components `Node` and `Query`, then arriving at the central orchestrator class `DecisionTree`.

### A. Data Handling

The central data format of the QDT is a modified dictionary `ProblemData`, gathering all information for the current run. It is populated by the nodes, typically loading the problem instance first and subsequently adding additional processed data, e.g., a QUBO matrix or QASM-encoded quantum circuit. Depending on the key of a particular entry, low-level data validation is performed, e.g., a *"qubo\_matrix"* entry should be a square two-dimensional array. The data is available in human-readable dictionary format, and nodes are required to indicate the entries they modify, which is essential in satisfying the transparency requirement.

### B. The Node class

The atomic building block of the QDT and central enabler of modularity and locality is the `Node`. Nodes for specific tasks are inherited from the abstract `Node` class with attributes in table I. When implementing a concrete `Node` for a specific task, the relevant `ProblemData` entries are defined, both the required ones and those that will be created or modified by

the node. The QDT configuration then defines the children within the concrete QDT instance. During execution, the QDT instance makes all required `ProblemData` available. Furthermore, dynamic information can be requested during execution when necessary (e.g., queue lengths).

At runtime, `execute` and `interpret_result` are invoked on the forward and backward passes respectively. For example, an encoding node translates an application-specific instance formulation to a QUBO format, and interprets the solution on the backward pass.

The modularization of the QDT makes branching decisions challenging since the children of the nodes are only defined in a specific QDT instance, and nodes should function in different QDT instances. Specialized branching nodes therefore can define a set of known children, and select the correct one in their `next_node` method. It is only invoked when a node has multiple children in a QDT instance. A typical example for branching are VQAs sharing parts of their setup such as the optimizer choice. A branching node can now realize non-local behavior. After the optimizer has been chosen, the algorithm setup and execution again depends on the particular VQA. A branching node will now read out the type of algorithm selected by an upstream node from `ProblemData`, and direct to the correct setup and execution nodes.

Table I: Methods and Attributes of the Node class.

| Method                        | Important Parameters                               | Description                                                                                                                |
|-------------------------------|----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <code>__init__</code>         | requires, creates, children                        | Executed on <code>DecisionTree</code> instantiation, defines necessary <code>ProblemData</code> entries and children.      |
| <code>execute</code>          | <code>problem_data</code> , <code>path_info</code> | Executed on the forward pass with the required problem data entries, and optional information to select a particular path. |
| <code>next_node</code>        |                                                    | Executed on the forward pass if the node has more than one child node.                                                     |
| <code>interpret_result</code> | <code>result</code> , <code>problem_data</code>    | Executed on the backward pass with the result and the relevant problem data entries.                                       |
| <code>request_info</code>     |                                                    | Pipeline to request data from the <code>DecisionTree</code> instance during runtime.                                       |

### C. The Query class

Configurable automation is enabled by the `Query` class. A global flag sets the behavior of all queries in all nodes: On manual mode, users will be prompted for input, e.g., for algorithm selection. A default choice or recommendation based on the problem data can still be given. `Query` subclasses exist for various input types (various number formats, file paths, single-choice selection, multiple input values). Additional versatility is provided by combining queries into a `QueryTree` where subsequent queries can be executed conditionally on previous input. In the automatic mode, user prompts are skipped. Instead, default options, or recommendations computed from the problem data, are chosen. Queries are therefore essential to the requirement of automation without restriction.

### D. The DecisionTree class

The overarching orchestration, data handling and execution is handled by the `DecisionTree` class whose important components are shown in table II. Its main functionalities are instance creation, configuration and execution.

Table II: Methods and Attributes of the DecisionTree class.

| Method/Attribute           | Important Parameters                              | Description                                                                                         |
|----------------------------|---------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <code>__init__</code>      | <code>config</code>                               | Creates an instance based on the provided configuration file.                                       |
| <code>validate</code>      |                                                   | Validates the setup during initialization.                                                          |
| <code>run</code>           | <code>problem_instance</code> , <code>path</code> | Execute a decision tree run (forward and backward pass) on the specified problem instance and path. |
| <code>load_backends</code> |                                                   | Retrieve all available backends from the specialized backend provider nodes.                        |

The instantiation of the `DecisionTree` class requires a configuration object in YAML file format or as a dictionary. It defines the source packages of concrete nodes (in the QDT core, this is `quast_decisiontree.nodes`), the names of the nodes, their children, and optionally arguments for their initialization. By defining children, the entire tree structure is fixed. The required structure is a directed acyclic graph (DAG). Part of a QDT instance is shown in fig. 2.

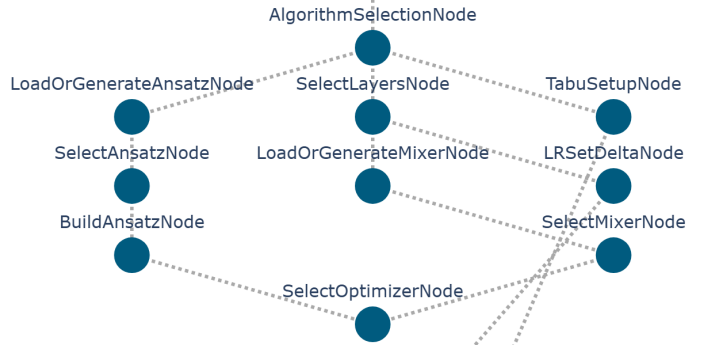


Figure 2: The algorithm selection part of a basic `DecisionTree` instance. Based on the selected algorithms, different choices follow: the definition of an ansatz (e.g., variational quantum eigensolver [49]), a depth and mixer (QAOA [23]), or skipping for a direct solver setup. Multiple nodes take part in the hyperparameter definitions. In the VQA case, the paths converge for the optimizer selection, which is not required in other cases (here: linear ramp QAOA [50] and a classical solver).

After instantiation, a validation step matches `ProblemData` entries between nodes to verify that the setup is functional.

The `DecisionTree` instance maintains a dictionary of QC backends using a specialized `BackendNode`. This enables retrieving current backend information and submitting quantum circuits through an external submission pipeline. The backend nodes are adapters to enable abstraction across different access modalities. During execution, all nodes can then request

information on available backends, with the `DecisionTree` instance requesting and forwarding the data.

Once the QDT instance is created, its structure (the nodes and their connections) is fixed, but the overall behavior can be configured – e.g., how detailed the logging output is, and to what extent decisions should be taken without user input. Thus, the `DecisionTree` satisfies the configurability requirement.

The `run` method then executes the `DecisionTree`. A problem instance and further path specification can be passed directly. The problem instance in JSON format needs to match a corresponding problem class as described in section VI-B, with a general QUBO class available next to application-specific instances. The path specification allows modifying the solution path taken through the decision tree, e.g., forcing an algorithm or setting a hyperparameter. To this end, node classes define *path keys*: E.g., a QAOA node will accept a key setting the depth of the ansatz.

On execution, the QDT iterates through the nodes, starting from the root node and invoking the `execute` methods, making matching entries of the problem data, path specification, and on request dynamic information, available to the node. After node execution, the modifications are written back to the `ProblemData`, and execution continues with the next node.

The forward pass finishes when a final node is reached, typically resulting in the execution of a hybrid algorithm. Once the result is available, the backward pass starts, iterating through the nodes in reverse for postprocessing, decoding and result interpretation. Similar to the `ProblemData` on the forward pass, a result dictionary is built on the backward pass. The backward pass then finishes with the root node which concludes the full run. In summary, the working of the QDT execution is shown in algorithm 1.

---

**Algorithm 1** QuaST Decision Tree

---

```

1: config ← load_config()
2: node ← init_root_node()
3: path ← []
4: problem_data ← init()
5: while node is not final do
6:   problem_data ← node.execute(problem_data)
7:   append node to path
8:   node ← node.next_node()
9: end while
10: result ← node.execute(problem_data)
11: for node in reverse(path) do
12:   result ← node.interpret_result(result)
13: end for

```

---

## VI. TEMPLATES, INPUT AND OUTPUT FORMATS

The `quast_decisiontree` package contains templates for backends and problem classes, as well as a `Builder` mechanic to handle difference in the interfaces of, e.g., optimizers. The QDT uses YAML for configuration and tree definition, and

JSON for problem instances and results. Run-specific and persistent logs are created automatically.

### A. Backends

At the most basic level, QC backends accept a quantum circuit, execute it, and return a set of measurement outcomes. They are typically embedded in a lower-level software stack handling compilation, optimization, and other pre- and postprocessing steps like error mitigation. The QDT aims to be device-agnostic and relies on specialized backend nodes to make the necessary data transformation to and from the backends. `DecisionTree` instances maintain a general list of available backends in a custom dictionary format, with mandatory specifications being a unique identifier and the provider node handling the backend communication. It is responsible for implementing the necessary data requests for specific information such as qubit counts, connectivity maps, calibration information etc.

### B. Problem Classes

CO problem classes are defined by a specification of how to describe problem instances, and a way to evaluate the objective function. The QDT template `OptimizationProblem` additionally demands a `formulate_problem` function used to translate the instance into a suitable format like QUBO. Problem instances are defined in JSON files, or equivalently Python dictionaries, with the application-specific definition in the `from_dict` method. Different formulation options can be included via formulation modes. The objective evaluation is then required to be independent of the formulation, to encourage a clear, application-specific performance metric. For instance, a TSP class defines city-coordinate input, QUBO translation with selectable encodings, and a path-length objective. This interface allows the QDT nodes to load a problem instance and transform it for further processing. Additional problem classes can be integrated without directly modifying the node loading the problem.

### C. Builders

The `Builder` template allows the QDT to efficiently access options that are not available behind a common interface. The prototypical case is the selection of an optimizer in VQA setup. Optimizers provided, e.g., by `scipy` [51], differ in the input to set them up: Gradient descent algorithms might require step lengths whereas genetic algorithms require a population size.

For these cases, the `Builder` class provides a unified wrapper. `HyperParam` objects define the required input. Hence, a node can dynamically provide a range of options by collecting all builders of a certain type, obtaining a list of hyperparameters from the builders, and setting the required hyperparameters via user prompt, class defaults, or by invoking an automation module.

Builders are central for achieving modularity and expandability: After a node for a specific task such as optimizer selection is implemented, the range of options it provides can

be expanded without modifying the node itself, simply by providing an appropriate builder instance. Currently, the core version uses builders for the creation of variational ansatzes according to a given template, and for optimizer selection.

## VII. USAGE SCENARIOS AND VALIDATION

The flexibility of the modular QDT is highlighted by the intended usage scenarios it can be adapted to, briefly described in this section. Initial validation is provided for two scenarios, with the commercial solver integration left as future work.

### A. Upper Part of an HPC Software Stack

Within an HPC software stack handling quantum circuit and algorithm submission, the following setup is required:

- A specialized backend provider node for circuit submission and retrieval of backend information.
- A well-defined QDT configuration from problem loading to the available algorithms and hyperparameter choices.
- A server infrastructure running the QDT instance, with submission formats mapping to the problem classes handled by the instance.

The QDT instance is then set up by an administrator, with free rein over the available algorithms, and what choices end users are allowed to make. From an end user perspective, only the preparation of the problem instance in JSON format and, optionally, the path file in YAML format is required. Then, the QDT instance translates the input into a format suitable for the lower HPC software stack, submits the circuits and processes the results. The end user profits from an *extension of software stack capabilities*: Instead of submitting quantum circuits, they can directly submit their problem instances, trusting the automation of the QDT to provide the best solution.

As a first validation, a local instance of the QDT is used to submit circuits for Linear-Ramp QAOA [50] to the Munich Quantum Portal, offering access to QC backends via the Munich Quantum Software Stack [47]. Overhead and postprocessing times for random QUBO problems are shown in table III. The QDT handles the encoding, preprocessing and decoding steps with minimal overhead when compared with the wait time for the results of circuit execution (which are dominated by the wait time between scheduling and execution of the jobs, even with an empty queue as in the experiment). Each instance was repeated 5 times.

Table III: Execution overhead (wall-clock times) on the submission of LR-QAOA circuits to the 20-qubit backend *EuroQExa20* via the Munich Quantum Portal

| QUBO size | Local QDT Forward Pass | Wait Time for Result | Local QDT Backward Pass |
|-----------|------------------------|----------------------|-------------------------|
| 4         | < 1 s                  | (11.8 ± 1.7) s       | < 1 s                   |
| 12        | < 2 s                  | (33.4 ± 1.7) s       | < 1 s                   |
| 20        | < 2 s                  | (39.4 ± 1.5) s       | < 1 s                   |

### B. Rapid Prototyping and Configuration

In research and development, the QDT can be used to quickly configure experiments. The circuits are either run by

the QDT instance itself or collected and submitted manually. The required setup is:

- A well-defined QDT configuration containing the necessary solution paths (e.g., different algorithms, encoding choices or hyperparameters).
- Nodes implementing the novel paths to be explored.
- Path files defining the experiment configurations, e.g. different encoding choices, a set of benchmarking instances or hyperparameter selections.

The researcher maintains full control over the QDT configuration and the experiment workflow. The QDT simplifies the modification of the experimental setup and allows reproducing experiments. Furthermore, a specialized researcher does not need to implement the entire solution pipeline from scratch, but can use existing QDT nodes. Researchers benefit from being able to investigate the full solution pipeline in an applied setting, rather than isolated components.

For illustration, table IV shows the approximate lines of code (LOC) to integrate a novel solution component under the assumption that the component is available as a standard Python object (class or function). Once implemented, the end-to-end pipeline can be directly executed by providing a file specifying a QDT path. The implementation effort is particularly low when **Builder** classes are used, since no nodes need to be modified.

Table IV: Estimated integration effort for new components.

| Novel Component                            | Tasks for Integration                                                                                                     | Effort (LOC) |
|--------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|--------------|
| Encoding for problem class                 | add new formulation mode to problem class, and adapters reading in problem class information                              | ~10          |
| Optimizer                                  | wrap optimizer in <b>Builder</b> class and define hyperparameters                                                         | ~5           |
| Ansatz for variational algorithm           | wrap ansatz generator in <b>Builder</b> class and define hyperparameters                                                  | ~5           |
| Quantum algorithm                          | implement nodes for hyperparameter selection, and algorithm setup                                                         | ~50          |
| Backend                                    | implement node for backend circuit submission and information retrieval                                                   | ~50          |
| Automation Module                          | read relevant information from problem data, insert into setup node, and characterize the selected option in problem data | ~50          |
| Custom pipeline for information extraction | node including readout from result data, and executing the custom pipeline                                                | ~20          |

### C. Translation Component for Industrial Solvers

Industrial solvers for optimization are deployed to end users in a customer-specific setup starting with data collection and preprocessing, then sending it to a commercial solver, and finally processing the result to provide the desired output. In this context, the QDT serves as a pre- and postprocessing accelerator for quantum-enhanced solutions. The manual overhead in deploying solutions for industrial optimization is reduced by including a QDT instance specialized in orchestrating hybrid quantum-enhanced solutions. Furthermore, this modularization makes already deployed solutions easily reusable and adaptable to new customers and solution pipelines. For providers

of industrial solvers, this drastically reduces the effort in deploying quantum-enhanced solutions to end users.

### VIII. AN AUTOMATION MODULE: SCALABILITY OF VARIATIONAL QUANTUM ALGORITHMS

The purpose of the QDT is to provide abstraction by automation. To illustrate this, the automation module described in this section implements data-driven recommendations for VQA's. At its core sits a database of shot requirements  $n_{\text{shots}}$  for various combinations of VQA and optimizer. Given a problem instance, the module then recommends the most resource-efficient combination, or provides an estimate for the user-selected configuration.

#### A. Methodology and Database

The module is based on the rigorous methodology for determining the measurement shot requirements of VQA's [17]. For a given VQA/optimizer combination, problem type, and QUBO density, it allows characterizing how the finite sampling error and the optimizer noise resilience threshold scale with system size, yielding a shot requirement curve  $n_{\text{shots}}(n)$  and a comparison against the quantum disadvantage boundary (see section II-C). The automation module uses a database with precomputed fit parameters, scaling exponents, and uncertainty estimates for the different configurations. The necessary data is obtained from precomputed benchmarking runs and can be continuously updated with new VQA/optimizer combinations and problem classes.

The database supports efficient queries, with the current coverage shown in table V. It can be continuously expanded and updated with new VQA variants, optimizers, problem classes, and combinations thereof. The extrapolated estimates to  $n = 100$  naturally carry substantial uncertainty. The propagated error bounds quantify this. An expansion of the data base is planned to reduce the uncertainties.

Table V: Current coverage of the scalability database. For each configuration,  $N = 100$  random instances are sampled to ensure statistical robustness of the scalability analysis.

| Category                            | Coverage                                                                                                                 |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| VQA variants                        | VQE with hardware-efficient ansatz [49], vanilla QAOA [23], warm-start QAOA [52], ma-QAOA [53], QAOA+ [54], DC-QAOA [45] |
| Classical optimizers                | NGD [55], COBYLA [56], SPSA [57], Powell [58], NFT [59]                                                                  |
| <i>Problem-specific parameters:</i> |                                                                                                                          |
| Random QUBO                         | density $\rho \in \{0.1, 0.2, 0.3, 0.5, 0.7, 1.0\}$                                                                      |
| MaxCut                              | density $\rho \in \{0.1, 0.2, 0.3, 0.5, 0.7, 1.0\}$                                                                      |
| Knapsack                            | capacity ratio $r \in \{0.2, 0.4, 0.5, 0.6, 0.8\}$                                                                       |
| System sizes (fitted)               | $n = 3$ to $10$                                                                                                          |
| Extrapolation range                 | $n = 10$ to $100$                                                                                                        |

#### B. Functional Module Description

**Input.** The module requires a QUBO matrix  $Q \in \mathbb{R}^{n \times n}$ , supplied directly or via the QUBO problem class of the QDT. Then, its size  $n$  (number of binary variables) and density (fraction of nonzero entries) are extracted.

Additionally, a problem class can be declared (e.g., Knapsack, MaxCut). In this case, the module uses the matching subset of the database to capture problem-specific scaling. Without a specified class, an automatic matching is performed based on size and density, using the general QUBO portion of the benchmarks. The user may then request either estimates for a specific VQA/optimizer combination or a recommendation among all available options.

**Output.** Module output is written to two files:

- 1) **scalability\_assessment.json** contains a structured analysis with the input problem characteristics, shot estimates including uncertainty bounds under the three scaling scenarios, and the indication whether the estimate is *feasible*, i.e., falls outside the quantum disadvantage regime. The VQA/optimizer combination with the lowest shot requirement is then recommended, if any fall below the quantum disadvantage threshold.
- 2) **recommended\_config.yaml** provides the QDT configuration specifying the necessary node configurations including algorithm selection, optimizer setup, circuit depth, and backend selection. Hyperparameters are set to the values used in benchmarking and database creation, ensuring consistency between predicted shot requirements and execution.

This dual output design lends itself to a fully automated operation mode where the configuration file is passed directly to the QDT, or manual inspection of analysis results.

#### C. Architecture and Integration

The module contains four components:

- The **QUBO Analyzer** extracts problem size and density from the input QUBO and handles problem type matching.
- The **Scaling Database** stores the scaling parameters obtained in previous benchmarking, organized by problem type, density, VQA variant, and optimizer.
- The **Shot Estimator** computes predictions of  $n_{\text{shots}}$  under the three scaling scenarios.
- The **Recommendation Engine** evaluates the results, ranks combinations, and generates output files.

**Integration modes.** Depending on the usage scenario, the module is integrated into the QDT with one or two nodes:

- 1) **Recommendation mode:** A node assessing scalability is inserted between problem encoding and algorithm selection. The QUBO matrix is read from **ProblemData**, then the VQA/optimizer combinations are evaluated. If feasible quantum combinations exist, the QDT proceeds with algorithm selection using the determined recommendations. If no quantum approach is feasible, the recommendation to choose a classical algorithm (or non-variational QC algorithm) is passed on.
- 2) **Estimation mode:** A node to estimate scalability is inserted after algorithm selection. The node computes estimates for the chosen combination, allowing users to make informed decisions about resource allocation.

#### D. Demonstration on MaxCut

As a demonstration, a MaxCut instance with  $n = 60$  variables and graph density  $\rho = 0.4$  is used. The mapping to QUBO is straightforward (see, e.g., [60]), therefore the module directly queries the database and computes shot estimates under three scaling hypotheses (exponential, power-law, logarithmic). To ensure robust recommendations, a conservative approach is adopted: for each configuration, the *worst-case*, i.e., highest, estimate among valid fits is reported and compared to the quantum disadvantage threshold.

Table VI shows the resulting assessment. The module classifies each configuration into one of three categories:

- **Feasible:** The worst-case shot estimate falls below the quantum disadvantage boundary.
- **Infeasible:** At least one scaling hypothesis yields a valid estimate, but all exceed the boundary.
- **Not characterizable:** The VQA-optimizer combination fails to produce consistent convergence behavior even at small system sizes, preventing extraction of meaningful scaling parameters under any hypothesis.

Table VI: Worst-case shot estimates for a 60-variable weighted MaxCut instance (density  $\rho = 0.4$ ). “n.c.” indicates non-characterizable configurations with inconsistent convergence.

| VQA                | Optimizer                 | $n_{\text{shots}}$    | Status     |
|--------------------|---------------------------|-----------------------|------------|
| VQE                | NGD                       | $1.9 \times 10^6$     | feasible   |
|                    | Powell                    | $6.5 \times 10^{12}$  | feasible   |
|                    | NFT                       | $1.9 \times 10^{15}$  | infeasible |
|                    | COBYLA, SPSA              | n.c.                  | –          |
| ma-QAOA<br>$p = 1$ | Powell                    | $5.8 \times 10^{14}$  | infeasible |
|                    | COBYLA                    | $1.4 \times 10^{22}$  | infeasible |
|                    | NGD                       | $8.3 \times 10^{51}$  | infeasible |
|                    | NFT, SPSA                 | n.c.                  | –          |
| QAOA+<br>$p = 1$   | Powell                    | $3.6 \times 10^{80}$  | infeasible |
|                    | NGD, NFT, SPSA, COBYLA    | n.c.                  | –          |
| DC-QAOA<br>$p = 5$ | Powell                    | $1.6 \times 10^{25}$  | infeasible |
|                    | COBYLA                    | $1.3 \times 10^{145}$ | infeasible |
|                    | NGD, NFT, SPSA            | n.c.                  | –          |
| QAOA<br>$p = 5$    | Powell                    | $9.1 \times 10^{19}$  | infeasible |
|                    | SPSA                      | $1.5 \times 10^{62}$  | infeasible |
|                    | COBYLA                    | $2.6 \times 10^{144}$ | infeasible |
|                    | NGD, NFT                  | n.c.                  | –          |
| WS-QAOA<br>$p = 5$ | NFT                       | $2.5 \times 10^{50}$  | infeasible |
|                    | NGD, SPSA, COBYLA, Powell | n.c.                  | –          |

Of all configurations evaluated, only VQE with the optimizers NGD and Powell produce feasible options. All QAOA-based variants, including vanilla QAOA, ma-QAOA, QAOA+, DC-QAOA, and WS-QAOA, either exhibit extreme shot requirements or cannot be characterized. Most QAOA variants

fail to produce characterizable scaling behavior, reflecting the fundamental challenge posed by the rugged optimization landscapes that prevent reliable convergence even at small system sizes, making extrapolation to larger instances infeasible. The module’s ability to identify and flag such cases is essential for trustworthy recommendations. Based on this assessment, the module recommends VQE with NGD as the optimal configuration and generates the corresponding QDT configuration. If no feasible configuration exists for a given problem instance, the module routes to a classical solver, preventing resource usage on quantum-enhanced approaches that cannot outperform brute-force search.

#### IX. DISCUSSION AND OUTLOOK

With the QDT, we present the concept and prototypical realization of a framework providing abstraction to end users of quantum-enhanced optimization. The central challenge is the realization of useful automation modules in a rapidly evolving, diverse environment. The QDT, therefore, is designed with maximum modularity and generality in mind to allow for *automation without restriction*. The design principle outlined in section III maximizes the usability of the QDT. Section IV and section V then describe the framework following a tree-like structure of computation nodes acting on a transparent problem data format. Thus, seemingly contradictory goals are accomplished: hiding complex decisions behind an abstraction layer for end users while allowing experts to overwrite recommendations and explore novel paths. The resulting framework is useful in different situations (section VII): in the context of an HPC software stack where it benefits end users by extending the software stack to their applications; as a tool for applied research and development, and as a translation layer to easily include quantum-enhanced solutions in commercial solvers.

A concrete automation module based on a robust VQA scalability analysis [17] shows how abstraction is provided by evaluating the problem-specific feasibility of VQA variants in conjunction with the classical optimizer.

With the QDT, quantum-enhanced solutions for CO are one step closer to adoption by end users. The design requirements lead to a framework useful to researchers and users alike. At the current maturity of the rapidly evolving field of applied QC research, this is crucial, for bridging the gap to industrial adoption.

The crucial next development step in the QDT is its validation at increasing technology readiness, moving towards productive environments. The envisioned usage scenarios (section VII) will lead to an iterative refinement and improvement of the framework. Additionally, further automation modules will be included in analogy to section VIII, or by integrating third-party software tools.

##### Note On Code Availability

The source code for the QuaST Decision Tree framework without the automation module described in section VIII will be provided as open source software in summer 2026.

# REFERENCES

- [1] J. M. Weinand, K. Sörensen, P. San Segundo, M. Kleinebrahm, and R. McKenna, “Research trends in combinatorial optimization,” *International Transactions in Operational Research*, vol. 29, no. 2, pp. 667–705, 2022.
- [2] N. Mazyavkina, S. Sviridov, S. Ivanov, and E. Burnaev, “Reinforcement learning for combinatorial optimization: A survey,” *Computers & Operations Research*, vol. 134, p. 105400, 2021.
- [3] A. Abbas et al., “Challenges and opportunities in quantum optimization,” *Nature Reviews Physics*, vol. 6, no. 12, pp. 718–735, 2024.
- [4] C. H. Bennett, E. Bernstein, G. Brassard, and U. Vazirani, “Strengths and Weaknesses of Quantum Computing,” *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1510–1523, 1997.
- [5] M. Cerezo et al., “Variational quantum algorithms,” *Nature Reviews Physics*, vol. 3, no. 9, pp. 625–644, 2021.
- [6] A. Montanaro, “Quantum speedup of branch-and-bound algorithms,” *Phys. Rev. Res.*, vol. 2, no. 1, p. 013056, 2020.
- [7] R. Seidel et al., *Qrisp: A Framework for Compilable High-Level Programming of Gate-Based Quantum Computers*, arXiv: 2406.14792, 2024.
- [8] Qiskit contributors, *Qiskit: An Open-source Framework for Quantum Computing*, Zenodo: 2573505, 2023.
- [9] A. Cross et al., “OpenQASM 3: A Broader and Deeper Quantum Assembly Language,” *ACM Transactions on Quantum Computing*, vol. 3, no. 3, 12:1–12:50, 2022.
- [10] QIR Alliance, *QIR Specification*, 2021. [Online]. Available: <https://github.com/qir-alliance/qir-spec>
- [11] N. Quetschlich, L. Burgholzer, and R. Wille, “MQT Predictor: Automatic Device Selection with Device-Specific Circuit Compilation for Quantum Computing,” *ACM Transactions on Quantum Computing*, vol. 6, no. 1, 2025.
- [12] R. Coelho, G. Kruse, and J. M. Lorenz, *QAOA-Predictor: Forecasting Success Probabilities and Minimal Depths for Efficient Fixed-Parameter Optimization*, arXiv:2603.02990 [quant-ph], 2026.
- [13] O. Di Matteo, S. Núñez-Corrales, M. Stęchły, S. P. Reinhardt, and T. Mattson, “An Abstraction Hierarchy Toward Productive Quantum Programming,” in *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, vol. 01, 2024, pp. 979–989.
- [14] *Iskay Quantum Optimizer – A Qiskit Function by Kipu Quantum*. [Online]. Available: <https://quantum.cloud.ibm.com/docs/en/guides/kipu-optimization>
- [15] *Optimization Solver – A Qiskit Function by Q-CTRL Fire Opal*. [Online]. Available: <https://quantum.cloud.ibm.com/docs/en/guides/q-ctrl-optimization-solver>
- [16] B. Poggel, X. Runge, A. Bärligea, and J. M. Lorenz, *Creating Automated Quantum-Assisted Solutions for Optimization Problems*, arXiv: 2409.20496, 2024.
- [17] A. Bärligea, B. Poggel, and J. M. Lorenz, “Scalability challenges in variational quantum optimization under stochastic noise,” *Phys. Rev. A*, vol. 112, no. 3, p. 032407, 2025.
- [18] G. Van Rossum and F. L. Drake, *Python 3 Reference Manual*. CreateSpace, 2009.
- [19] A. Lucas, “Ising formulations of many NP problems,” *Frontiers in Physics*, vol. 2, 2014.
- [20] M. Schnaus et al., “Efficient Encodings of the Travelling Salesperson Problem for Variational Quantum Algorithms,” in *2024 IEEE International Conference on Quantum Software (QSW)*, 2024, pp. 81–87.
- [21] M. Schmid, S. Braun, R. Sollacher, and M. J. Hartman, “Highly efficient encoding for job-shop scheduling problems and its application on quantum computers,” *Quantum Science and Technology*, vol. 10, no. 1, p. 015051, 2025.
- [22] L. K. Grover, “A Fast Quantum Mechanical Algorithm for Database Search,” in *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, 1996, pp. 212–219.
- [23] E. Farhi, J. Goldstone, and S. Gutmann, “A quantum approximate optimization algorithm,” *arXiv preprint arXiv:1411.4028*, 2014.
- [24] K. Blekos et al., “A review on Quantum Approximate Optimization Algorithm and its variants,” *Physics Reports*, vol. 1068, pp. 1–66, 2024.
- [25] J. R. McClean, S. Boixo, V. N. Smelyanskiy, R. Babush, and H. Neven, “Barren plateaus in quantum neural network training landscapes,” *Nature Communications*, vol. 9, no. 1, p. 4812, 2018.
- [26] E. R. Anschuetz and B. T. Kiani, *Beyond Barren Plateaus: Quantum Variational Algorithms Are Swamped With Traps*, arXiv: 2205.05786, 2022.
- [27] IBM Quantum, *IBM Quantum Roadmap*. [Online]. Available: <https://www.ibm.com/quantum/hardware#roadmap>
- [28] *IonQ | Roadmap*. [Online]. Available: <https://www.ionq.com/roadmap>
- [29] *Roadmap – IQM Quantum Computers*, 2025. [Online]. Available: <https://meetiqm.com/technology/roadmap/>
- [30] A. Gilliam, S. Woerner, and C. Gondiulea, “Grover Adaptive Search for Constrained Polynomial Binary Optimization,” *Quantum*, vol. 5, p. 428, 2021.
- [31] *The Advantage™ Quantum Computer | D-Wave*. [Online]. Available: <https://www.dwavesys.com/solutions-and-products/systems/>
- [32] *Functions | IBM Quantum Platform*. [Online]. Available: <https://quantum.cloud.ibm.com/functions>
- [33] A. Cobb, J.-G. Schneider, and K. Lee, “Towards Higher-Level Abstractions for Quantum Computing,” in *Proceedings of the 2022 Australasian Computer Science Week*, 2022, pp. 115–124.

- [34] H. Fürntratt, P. Schnabl, F. Krebs, R. Unterberger, and H. Zeiner, “Towards Higher Abstraction Levels in Quantum Computing,” in *Service-Oriented Computing – ICSOC 2023 Workshops*, 2023, pp. 162–173.
- [35] O. Di Matteo, “The Art of Abstraction in Quantum Software,” in *2025 IEEE/ACM International Workshop on Quantum Software Engineering (Q-SE)*, 2025, pp. 25–26.
- [36] B. Heim et al., “Quantum programming languages,” *Nature Reviews Physics*, vol. 2, no. 12, pp. 709–722, 2020.
- [37] A. Zulehner and R. Wille, *Introducing Design Automation for Quantum Computing*. Springer, 2020.
- [38] M. Soeken, T. Haener, and M. Roetteler, “Programming quantum computers using design automation,” in *2018 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2018, pp. 137–146.
- [39] R. Wille et al., “The MQT Handbook: A Summary of Design Automation Tools and Software for Quantum Computing,” in *2024 IEEE International Conference on Quantum Software (QSW)*, 2024, pp. 1–8.
- [40] D. Volpe, N. Quetschlich, M. Graziano, G. Turvani, and R. Wille, “A Predictive Approach for Selecting the Best Quantum Solver for an Optimization Problem,” in *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, vol. 01, 2024, pp. 1014–1025.
- [41] D. Volpe, N. Quetschlich, M. Graziano, G. Turvani, and R. Wille, “Towards an Automatic Framework for Solving Optimization Problems with Quantum Computers,” in *2024 IEEE International Conference on Quantum Software (QSW)*, 2024, pp. 46–57.
- [42] D. Eichhorn, M. Schweikart, N. Poser, F. Fiand, B. Poggel, and J. M. Lorenz, “Hybrid Meta-Solving for Practical Quantum Computing,” in *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, vol. 01, 2024, pp. 421–431.
- [43] B. Poggel, N. Quetschlich, L. Burgholzer, R. Wille, and J. M. Lorenz, “Recommending Solution Paths for Solving Optimization Problems with Quantum Computing,” in *2023 IEEE International Conference on Quantum Software (QSW)*, 2023.
- [44] N. Sachdeva et al., *Integrated error-suppressed pipeline for quantum optimization of nontrivial binary combinatorial optimization problems on gate-model hardware at the 156-qubit scale*, arXiv:2406.01743 [quant-ph], 2026.
- [45] P. Chandarana et al., “Digitized-counterdiabatic quantum approximate optimization algorithm,” *Physical Review Research*, vol. 4, no. 1, p. 013 141, 2022.
- [46] P. R. Hegde et al., “Beyond the Buzz: Strategic Paths for Enabling Useful NISQ Applications,” in *Proceedings of the 21st ACM International Conference on Computing Frontiers*, 2024, pp. 310–313.
- [47] L. Burgholzer et al., “The Munich Quantum Software Stack: Connecting End Users, Integrating Diverse Quantum Technologies, Accelerating HPC,” in *Proceedings of the Supercomputing Asia and International Conference on High Performance Computing in Asia Pacific Region*, 2026, pp. 55–67.
- [48] *DecisionTree*. [Online]. Available: <https://www.quast-decisiontree.com/>
- [49] A. Peruzzo et al., “A variational eigenvalue solver on a photonic quantum processor,” *Nature Communications*, vol. 5, no. 1, p. 4213, 2014.
- [50] J. A. Montañez-Barrera and K. Michielsen, “Toward a linear-ramp QAOA protocol: Evidence of a scaling advantage in solving some combinatorial optimization problems,” *npj Quantum Information*, vol. 11, no. 1, p. 131, 2025.
- [51] P. Virtanen et al., “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020.
- [52] D. J. Egger, J. Mareček, and S. Woerner, “Warm-starting quantum optimization,” *Quantum*, vol. 5, p. 479, 2021.
- [53] R. Herrman, P. C. Lotshaw, J. Ostrowski, T. S. Humble, and G. Siopsis, “Multi-angle quantum approximate optimization algorithm,” *Scientific Reports*, vol. 12, no. 1, p. 6781, 2022.
- [54] M. Chalupnik, H. Melo, Y. Alexeev, and A. Galda, *Augmenting QAOA Ansatz with Multiparameter Problem-Independent Layer*, 2022. arXiv: 2205.01192 [quant-ph].
- [55] N. Kuete Meli, F. Mannel, and J. Lellmann, “A universal quantum algorithm for weighted maximum cut and ising problems,” *Quantum Information Processing*, vol. 22, no. 7, p. 279, 2023.
- [56] M. J. Powell, “A direct search optimization method that models the objective and constraint functions by linear interpolation,” in *Advances in Optimization and Numerical Analysis*, Springer, 1994, pp. 51–67.
- [57] J. Spall, “Multivariate stochastic approximation using a simultaneous perturbation gradient approximation,” *IEEE Transactions on Automatic Control*, vol. 37, no. 3, pp. 332–341, 1992.
- [58] M. J. Powell, “An efficient method for finding the minimum of a function of several variables without calculating derivatives,” *The Computer Journal*, vol. 7, no. 2, pp. 155–162, 1964.
- [59] K. M. Nakanishi, K. Fujii, and S. Todo, “Sequential minimal optimization for quantum-classical hybrid algorithms,” *Phys. Rev. Res.*, vol. 2, p. 043 158, 2020.
- [60] F. Barahona, M. Jünger, and G. Reinelt, “Experiments in quadratic 0–1 programming,” *Mathematical Programming*, vol. 44, no. 1, pp. 127–137, 1989.